KEEPING STUDENTS FOCUSED ONLINE: AN AI VIRTUAL CLASSROOM FOR ATTENTION MONITORING AND LEARNING GUIDANCE

(IntelliClass AI)

Session 2025 - 2027

Supervisor
Dr. Hannan Bin Liaqat, Associate Professor
Program: BS Computer Science (5th Semester)
Shift: Evening

Submitted By

1. Muhammad Musa, mtf25005277
2. Muhammad Ahmad, mtf25005249
3. Faraz Hassan, mtf25005258
4. Sohaib Khan, mtf25005265

DEPARTMENT OF INFORMATION SCIENCES
UNIVERSITY OF EDUCATION, LAHORE
CAMPUS Township Campus.

October 7, 2026

Declaration

This is to certify that, we declare that our FYP “Keeping Students Focused Online: An AI Virtual Classroom for Attention Monitoring and Learning Guidance” is the result of our own research and development. We have not copied the proposal from anywhere. We declare that the similarity index of this proposal, as verified through the university’s Turnitin check arranged by our supervisor before submission, is below 20%.

Submission Date: _____

Signature: _____
Name: Muhammad Ahmad
Date: October 7, 2026

Signature: _____
Name: Muhammad Musa
Date: October 7, 2026

Signature: _____
Name: Faraz Hassan
Date: October 7, 2026

Signature: _____
Name: Muhammad Sohaib Khan
Date: October 7, 2026

Table of Contents

1 Introduction..... 4

2 Related Work..... 4

3 Problem Description..... 7

4 Aims and Objectives..... 7

5 Domain of the Project..... 8

6 Project Scope..... 8

7 Methodology..... 9

8 Tools and Technology..... 13

9 Mile Stone (Gantt Chart)..... 14

10 References..... 15

1. Introduction

Most of our own classes at the University of Education can now be taught online, and the same is true across Pakistan and much of the world. A lecturer opens Zoom, Google Meet or Microsoft Teams and forty, fifty, sometimes sixty students join from their homes. These tools were built for business meetings. They carry the lecturer's voice and slides outward and bring a grid of small video tiles back, and that is where their understanding ends. Nothing in them knows whether the person behind a tile is awake.

A teacher in a physical room reads the class without thinking about it. Someone nodding off in the back row, someone staring out of the window, someone typing under the desk: each is noticed within seconds and answered with a question, a pause or a change of pace. Online, that loop is cut. The lecturer is talking, driving the slides and glancing at the chat, while the students are a crowded wall of thumbnails that nobody can really read. A student can fall asleep, walk away, stare at a second monitor or switch to a game, and the first sign the lecturer gets is a poor quiz result some weeks later.

We propose to put that missing feedback back, with a system we call **IntelliClass AI**. It is a complete web classroom, not a plug-in for somebody else's platform: the instructor creates a class, uploads the slides and teaches over live audio and video, and students sign in with their own accounts and roll numbers. On top of that, and only after a student has explicitly agreed, the student's own webcam and shared screen are analysed. Vision models estimate whether a face is present, whether the eyes are open, whether the student looks drowsy, and roughly where the head and gaze are pointing. A second pipeline reads what is on the shared screen with OCR and text embeddings and asks one plain question: does this still look like the slide being taught right now?

Two decisions run through the whole design. The first is that the face analysis happens **inside the student's own browser**. Raw video is never uploaded, never stored, never even seen by our server; what travels is a few bytes of numbers each second. That is good for privacy, and it also means the server can carry a large class on free hosting because it is not decoding anyone's camera. The second decision is that no judgement is ever made from a single frame. An **alert engine** weighs how confident the model is against how long a condition has lasted, then applies cooldown and grouping rules on top. A blink, or a two-second look down at a notebook, produces nothing. What the instructor receives is a short, ranked list of alerts worth acting on rather than a stream of noise.

The system keeps helping after the alert, too. Lecture audio is transcribed with an open-source speech model; summaries, key topics and likely exam questions are generated from the transcript; and a retrieval-augmented assistant answers students' questions from that lecture's slides and transcript only, saying plainly when the answer is not in the material. Instructors watch a live dashboard while they teach and get an analytics report once the class ends.

In practice the project brings computer vision, machine learning and deep learning, natural language processing, generative AI and a real-time distributed back end together in one working product. We will judge every part of it with numbers: precision, recall and F1-score for the models, alert latency, resource use, and behaviour with up to one hundred students connected at once.

2. Related Work

Four areas of existing work matter here: the platforms used for online classes, research on recognising student engagement automatically, work on drowsiness and gaze estimation, and the tools now available for understanding lecture content. We take each in turn and close with the gap we intend to fill.

2.1 Online Meeting and Classroom Platforms

Zoom [1], Google Meet [2] and Microsoft Teams [3] carry most online teaching today. All three offer dependable audio and video, screen sharing, chat and recording, and some add attendance logs or simple participation counts such as time joined or messages posted. What none of them attempts is to tell whether a student is awake, in front of the camera, or looking at the lecture at all. Spotting disengagement is left to the instructor, and that is precisely the job that stops being possible once a class grows past a couple of dozen tiles.

2.2 Online Examination Proctoring Systems

Proctoring products such as Proctorio [4] watch a candidate's webcam and screen for the length of an exam and flag things like an absent face, a second person in the room or a switch to another window. They prove that these signals can be processed automatically. Their purpose, though, is almost the opposite of ours. They exist for short, high-stakes exams, not for everyday lectures; they record the whole session; and what they produce is an accusation. Coghlan, Miller and Paterson (2021) [5] argue that this kind of opaque, heavily recorded surveillance wears away the trust between students and their institution. We have taken the other road: alerts are supportive, nothing is recorded, and the system never labels anyone dishonest.

2.3 Automatic Engagement Recognition Research

Recognising engagement from a student's face is a reasonably mature research problem. Whitehill et al. (2014) [6] showed a decade ago that classifiers trained on facial expressions could judge engagement about as consistently as human labellers, which settled the question of whether the task is possible at all. Gupta et al. (2016) [7] then released DAiSEE, roughly nine thousand short clips of 112 people watching educational video on their own devices, each clip rated for boredom, confusion, engagement and frustration. Kaur et al. (2018) [8] went a step further by estimating engagement intensity and locating where in an unconstrained video it changes. The review by Dewan, Murshed and Lin (2019) [9] is the most useful summary of the field for our purposes, because two of its findings are the motivation for this project: almost all systems are evaluated **offline on recorded data**, and very few have ever been wired into a teaching platform that an instructor could use during an actual lecture.

2.4 Drowsiness Detection

Most drowsiness research comes from driver monitoring. Soukupová and Čech (2016) [10] proposed the Eye Aspect Ratio (EAR), a single number computed from six landmarks around each eye that drops sharply when the eyelid closes, and showed that it can detect blinks in real time on an ordinary computer. EAR is usually paired with PERCLOS, the share of time the eyes spend closed, and with blink rate; together they form the baseline that learned models are measured against. Public datasets such as the MRL Eye Dataset, Closed Eyes in the Wild and YawDD exist for training. We use EAR as our baseline as well, but the setting is different from a car: the camera is a laptop webcam at desk height, the lighting is whatever the student's room happens to have, and the observation runs for an hour or more.

2.5 Gaze Estimation and Facial Landmark Tracking

Zhang et al. (2015) [11] built MPIIGaze by collecting more than two hundred thousand face images from fifteen people on their own laptops over several months, and used it to show that appearance-based gaze estimation holds up under everyday lighting and head movement, while also documenting how it degrades at extreme angles and in poor light. MediaPipe (Lugaresi et al., 2019) [12] and its Face Mesh model (Kartynnik et al., 2019) [13] then made it practical to track 468 facial landmarks in real time on laptops, phones and, importantly for us, inside a web browser. Together these results make on-device attention estimation realistic. They also carry a

warning that we have built into the design: when the image is poor, an honest system should report an **uncertain** state rather than guess.

2.6 Lecture-Content and Document Understanding

The pieces needed to understand what is on a screen are now mature and open source. PP-OCR (Du et al., 2020) [14] recognises text with models only a few megabytes in size. CLIP (Radford et al., 2021) [15] maps images and text into one embedding space, and Sentence-BERT (Reimers & Gurevych, 2019) [16] gives fast semantic similarity between passages of text. Whisper (Radford et al., 2023) [17], trained on roughly 680,000 hours of multilingual audio, handles speech-to-text in noisy conditions, and retrieval-augmented generation (Lewis et al., 2020) [18] lets a language model answer from a fixed set of documents instead of from memory. Each of these has been used many times on its own. We have not found published work that combines them to compare a student's live screen with the slide the instructor is presenting at that moment.

2.7 Classroom Automation Systems

A separate family of systems automates the administration of a **physical** classroom. Chen et al. (2020) [19] review the rapid growth of artificial intelligence in education and observe that most deployed systems serve administration, and only a few look at learning as it happens minute by minute. The typical project mounts one camera in the room, adds a mobile or desktop app, and automates attendance by face recognition, quiz creation and hosting, record keeping and note preparation, sometimes with an engagement report built from a photo taken every few minutes. Useful as they are, these systems assume everyone is in one room under one camera. They cannot follow an individual's eye state continuously, and they have no view of what is on each student's own screen. Their reports arrive periodically and serve administration; our interest is in what happens second by second during a remote lecture, and in reacting while it is still going on.

2.8 Research Gap

Put side by side, the work above leaves a visible hole:

- Meeting platforms deliver the class but learn nothing about the students in it, while proctoring tools read the same signals only for exams and only with full recording.
- Engagement research is accurate but offline and tied to datasets; drowsiness and gaze methods are mature but tuned for drivers, not learners.
- Classroom automation serves physical rooms and administrative chores, not individual remote students.
- None of the reviewed systems compares a student's live screen with the current slide, and none fuses these signals through confidence- and persistence-based alerting.

Table 1 sets the comparison out. Every source cited in this section appears in Section 10 with a paper or product link, so the originals are a click away.

Capability	Meeting platforms	Proctoring tools	Engagement research	Classroom automation	Proposed system
Delivers the live online class	Yes	No	No	No	Yes
Per-student webcam analysis	No	Yes	Yes	No (room camera)	Yes
Drowsiness (eye closure) detection	No	Partial	Partial	No	Yes

Capability	Meeting platforms	Proctoring tools	Engagement research	Classroom automation	Proposed system
Student screen vs. current slide	No	Partial	No	No	Yes
Reacts during the lecture	No	Yes	No (offline)	Periodic	Yes
Confidence + persistence alerting	No	No	No	No	Yes
On-device analysis, no video stored	N/A	No	N/A	No	Yes

Table 1: Comparison of existing solutions with the proposed system

3. Problem Description

A lecturer running a large online class is asked to do two things at once: teach, and judge whether dozens of remote students are still with the lecture. With every student reduced to a small tile, the second task is close to impossible. Students doze, step away from the camera, stare elsewhere for minutes, or drift to unrelated content on their own screens, and because the lecturer is busy presenting, these lapses surface only when assessment results come back poor.

The tools that exist each cover a fragment. Meeting platforms give communication and no sense of attention. Proctoring software does watch students, but it is built for exams, keeps full video and raises real privacy concerns. Engagement research produces accurate models that are tested offline on recordings. Classroom automation targets physical rooms and tasks such as attendance and quizzes. A naive automated fix would introduce problems of its own: deciding from single frames floods the instructor with false alarms, streaming every camera to a server neither scales nor respects privacy, and poor lighting or an odd camera angle makes any prediction shaky.

What is needed is a virtual classroom that works in real time and respects privacy; that turns permitted camera, screen and audio signals into a small number of trustworthy, ranked alerts while the lecture is still running; that keeps working when some signals are missing or unreliable; and that is there to support learning, not to accuse anyone.

4. Aims and Objectives

Aim

- To build an easy-to-use online classroom that quietly notices when a student has become sleepy, distracted or has drifted away from the lecture, tells the teacher only when it actually matters, and turns every lecture into notes and answers students can revise from, without recording or storing any video at all.

Objectives

1. **Build the classroom itself.** A website where a teacher starts a class and students join with their roll number, see and hear the teacher, and follow the slides as they are taught.
2. **Notice sleepy students.** Use the student's own camera to tell when the eyes stay shut too long, and report drowsiness only once it has gone on for several seconds. Target: correct in at least 85 out of 100 test cases.
3. **Notice when attention drifts.** Detect a student who has looked away from the lecture for a long stretch or has left the camera, and say "not sure" honestly when the light or the camera angle makes the call unsafe.

4. **Check that the student is on the right screen.** With the student's permission, compare what is open on the screen against the slide being taught, and report a mismatch only if it outlasts a short grace period.
5. **Send few alerts, and make them count.** Never alert on a blink or a two-second glance, and group repeated problems, so the teacher sees a short list and not a flood. Target: an alert reaches the teacher within 3 seconds.
6. **Let the teacher reach one student privately.** A tap on an alert plays a soft sound and shows a short message on that student's device alone, so nobody else is disturbed and nobody is embarrassed in front of the class.
7. **Give the teacher a live view and a report afterwards.** Show who needs attention right now during the lecture, and build a summary report automatically once it ends.
8. **Report with signals, never pictures.** The teacher receives only short status words per student, such as *sleepy*, *camera off*, *no face in front of the camera*, *looking away*, *wrong screen* or *connection problem*. No video, photograph or screenshot is ever sent, so the teacher sees at a glance who needs attention while no image of any student leaves that student's own device.
9. **Give each student a private summary of their own.** Every student can see their own attention and participation record and nobody else's, so the system helps them improve instead of merely reporting on them.
10. **Turn the lecture into notes.** Transcribe the teacher's voice, produce a short summary with the key points and likely questions, and let students ask questions that are answered only from that same lecture.
11. **Protect student privacy.** Run the face analysis on the student's own laptop, store no video or screen recording, tell students exactly what is collected, and keep a log of who viewed their information.
12. **Start simple, then prove the improvement.** Get a basic version of each feature working first, improve it, and show with measured results that the improved version really is better.
13. **Test it properly.** Evaluate with our own recorded samples, with up to 100 students connected at the same time, and with real teachers and students who use it and tell us what they think.

5. Domain of the Project

This is a **development project** in the domain of **Artificial Intelligence–based web application development** for **educational technology (EdTech)**. It draws on Computer Vision, Machine Learning and Deep Learning, Natural Language Processing and Generative AI, and joins them to real-time communication over WebRTC inside a containerised, service-oriented cloud architecture.

6. Project Scope

What we will deliver is a working web-based classroom for live online lectures with: on-device vision analysis (face presence, eye state and drowsiness, gaze and head direction); screen-to-slide compliance checking; a confidence- and persistence-based alert engine; a private one-to-one nudge from the instructor to a single student; an instructor dashboard with live alerts and post-class analytics; lecture transcription with summaries and a grounded assistant; and a containerised deployment evaluated with up to one hundred concurrent participants.

The following aspects are outside the scope of this project:

- Examination proctoring or any automatic decision about cheating or misconduct. The system raises alerts only; every decision stays with the instructor.
- Administrative classroom automation such as quiz generation, examination hosting, timetable or fee management. Attendance falls out of presence detection as a by-product and is not a module of this project.
- Emotion recognition, and identity verification based on face recognition. Students are identified by their authenticated account and roll number.
- Recording or storing raw video or continuous screen recordings. Only event metadata and the instructor's lecture material are stored.
- Operating-system level screen locking or hiding browser permission indicators, which a web application cannot do and should not try to.
- Matching commercial meeting applications feature for feature (breakout rooms, whiteboards, cloud recording), or native mobile applications.

Assumptions: students attend on a laptop or desktop with a webcam and a current Chromium-based browser; they consent explicitly to camera and screen sharing; the instructor uploads the lecture slides (PDF/PPT) before class; the internet connection is good enough for a video call; and lectures are delivered mainly in English, with some Urdu mixed in, which the speech-to-text model supports.

7. Methodology

We follow an **Agile incremental model** with two-week sprints. Each sprint ends with a working increment that is shown to the supervisor, so corrections arrive early instead of piling up at the end. For every AI component we build the plain baseline first, measure it, and only then replace or extend it with a learned model. That way each claimed improvement has a number behind it.

7.1 Classroom Platform and User Management

- Instructors and students register and sign in; JWT authentication and role-based access control guard every API endpoint.
- Instructors create classes, enrol students by roll number and upload the slides that later act as the reference for screen compliance.
- Before joining, each student sees a plain-language notice of which signals are analysed and grants camera, microphone and screen permissions one by one.
- Audio and video travel over WebRTC through a Selective Forwarding Unit, which scales much better than peer-to-peer connections. Only the instructor publishes video; a student's camera feeds the local vision worker and is never transmitted. Events and alerts use a separate secure WebSocket channel served by an in-process event bus, with Redis pub/sub added only if the API ever has to run as several replicas.

7.2 On-Device Vision Pipeline for Drowsiness and Attention

The first objective lives entirely in the student's browser, built on MediaPipe Face Mesh (Kartynnik et al., 2019) [13] running inside a Web Worker so that it keeps working when the class tab is in the background. Camera frames are sampled at five to ten per second and 468 landmarks are extracted from each. From those we compute the Eye Aspect Ratio (Soukupová & Čech, 2016) [10], blink rate and PERCLOS for drowsiness, and head pose together with gaze direction for attention. Nothing but these numbers leaves the device. On the server, a temporal classifier looks at a sliding window of them to decide the student's state. A blink or a quick glance is therefore not mistaken for drowsiness or distraction, and the classifier reports an *uncertain* state whenever lighting or camera angle make the estimate unreliable. We will

compare the rule-based EAR baseline against a learned classifier trained on public datasets such as the MRL Eye Dataset, YawDD and DAiSEE (Gupta et al., 2016) [7], together with a small test set that we record ourselves, with consent.

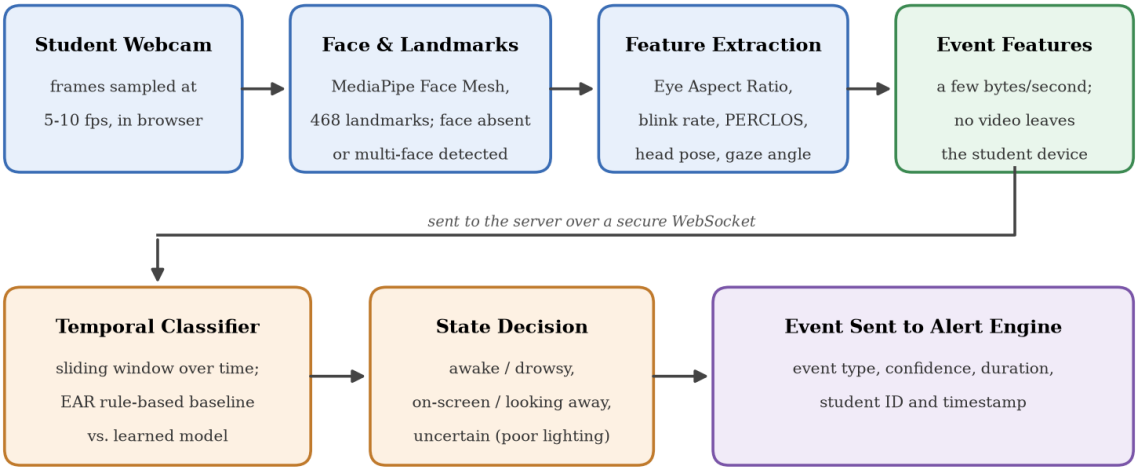


Figure 1: On-device vision pipeline for drowsiness and attention detection

7.3 Screen-Content Compliance and Slide Matching

The second objective asks whether what the student sees matches what is being taught. The baseline needs no model at all: the browser reports whether the class tab is still visible and in front, which already catches a switch to another window or application without taking a single screenshot. The content comparison is the extension built on top of it. The shared screen is sampled every few seconds, and a perceptual-hash filter drops frames that have not changed, which removes most of the processing cost. Frames that did change go through OCR (Du et al., 2020) [14] and image/text embeddings (Radford et al., 2021; Reimers & Gurevych, 2019) [15], [16], and the result is compared with the embedding of the instructor's current slide and the slides on either side of it. Scrolling, a page still loading, or opening a reference will all cause a short mismatch, so a low similarity score has to persist beyond a configurable grace period before a compliance event is raised.

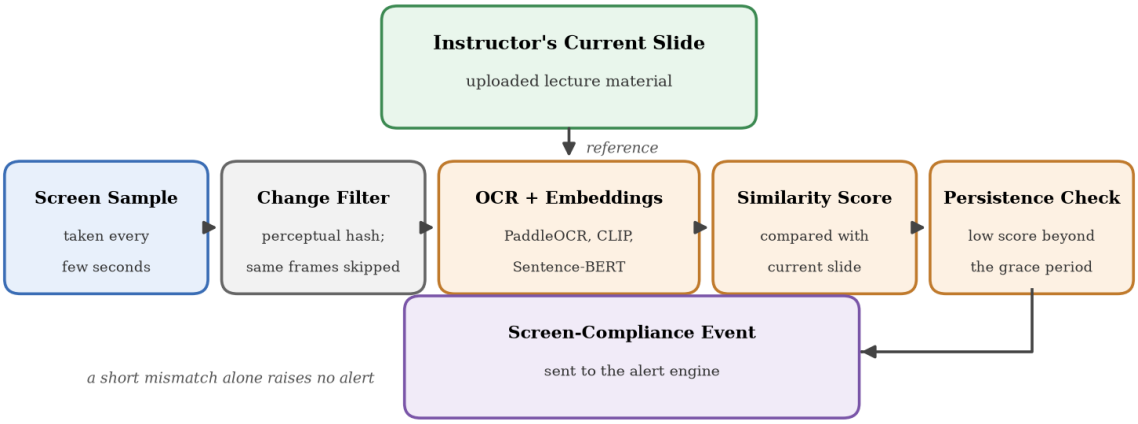


Figure 2: Screen-content compliance and slide-matching pipeline

7.4 Intelligent Alert Engine

The third objective is the part that turns raw detections into something an instructor can use. Each event carries the student identifier, event type, timestamp, confidence, duration and the module that produced it. An alert is raised only when three conditions hold at once: the

confidence is above a threshold, the condition has persisted longer than a configurable duration (drowsiness beyond five seconds, say, or looking away beyond twenty), and the same alert is not inside its cooldown window. Repeated events are grouped, which stops one struggling student from flooding the dashboard, and alerts are ordered by severity. We will tune every threshold on a validation set and report the values, rather than pick them by feel.

The engine also provides a **private nudge**. When the instructor taps an alert, a short message and a soft sound go out over the WebSocket channel of that one student, who is quietly pulled back to the lecture while the rest of the class hears and sees nothing. The nudge is addressed to a single session identifier and never broadcast to the room; it is rate-limited, which keeps it from being abused; and each one is written to the audit log with its sender, receiver and time. The student sees who sent it and can answer with one tap, “I am back” or “I have a connection problem”. That reply is stored as feedback on the alert and later used to measure how often the alerts were justified.

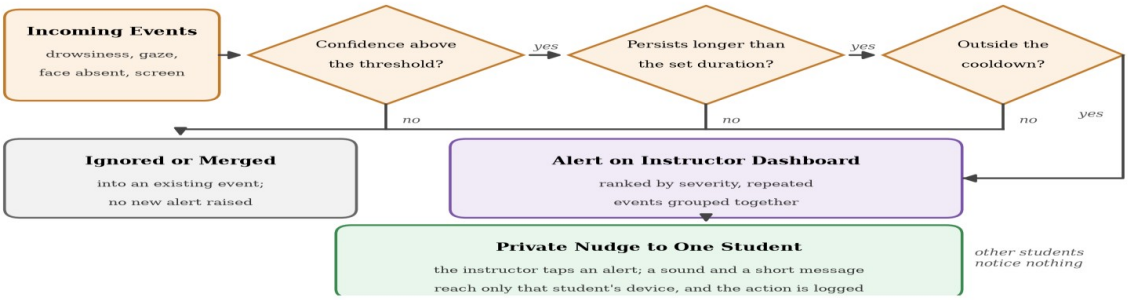


Figure 3: Alert engine decision flow and the private nudge to a single student

The dashboard is made of these status signals and nothing else. No video frame, photograph or screenshot of a student is ever sent to the instructor or kept on the server. The instructor sees a name, a roll number and a short state, as listed in Table 2.

Signal Shown	Meaning for the Instructor	Typical Trigger
Attentive	Present and following the lecture	Face detected and gaze towards the screen
Sleepy	The eyes have stayed closed for too long	EAR/PERCLOS past the threshold for several seconds
Looking away	Attention has been off the lecture for a while	Gaze or head turned away longer than the set duration
No face	Nobody is in front of the camera	No face detected for longer than the set duration
Camera off	The camera is switched off or blocked	No camera stream, or permission withdrawn
Wrong screen	What is visible no longer matches the slide	Low similarity beyond the grace period
Uncertain	Conditions are too poor to judge fairly	Weak landmark confidence, poor light or extreme angle
Connection problem	The student is losing the class, not ignoring it	Dropped or unstable real-time connection

Table 2: Status signals reported to the instructor (no video or images are sent)

7.5 Lecture Transcription and Learning Guidance

Lecture audio is transcribed close to real time with the open-source Whisper model (Radford et al., 2023) [17] running through faster-whisper. The transcript is split by topic, and a small open-

source language model writes summaries, key points and likely examination questions. Slides and transcript chunks are embedded and stored in a vector database. A retrieval-augmented assistant (Lewis et al., 2020) [18] can then answer questions from that lecture's material alone, citing the slide it drew on and saying that a topic was not covered when the retrieved context does not contain it.

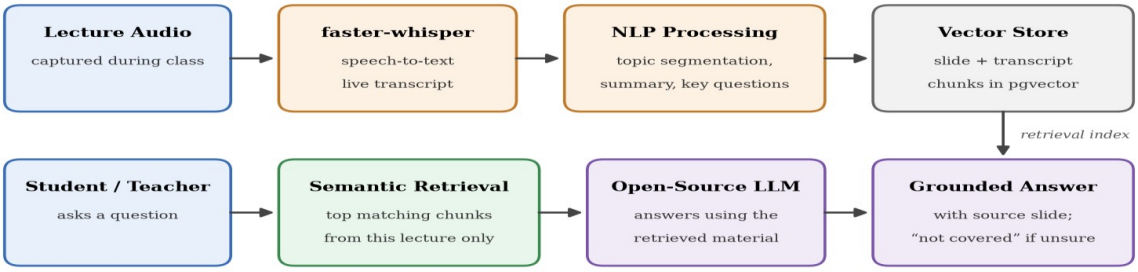


Figure 4: Lecture transcription, summarisation and retrieval-augmented assistance

7.6 System Architecture

Figure 5 shows how the parts fit together. Student and instructor browsers talk to the real-time layer and the application API; the AI services emit events that pass through the alert engine to the instructor dashboard; and only structured data, never raw video, reaches the data layer.

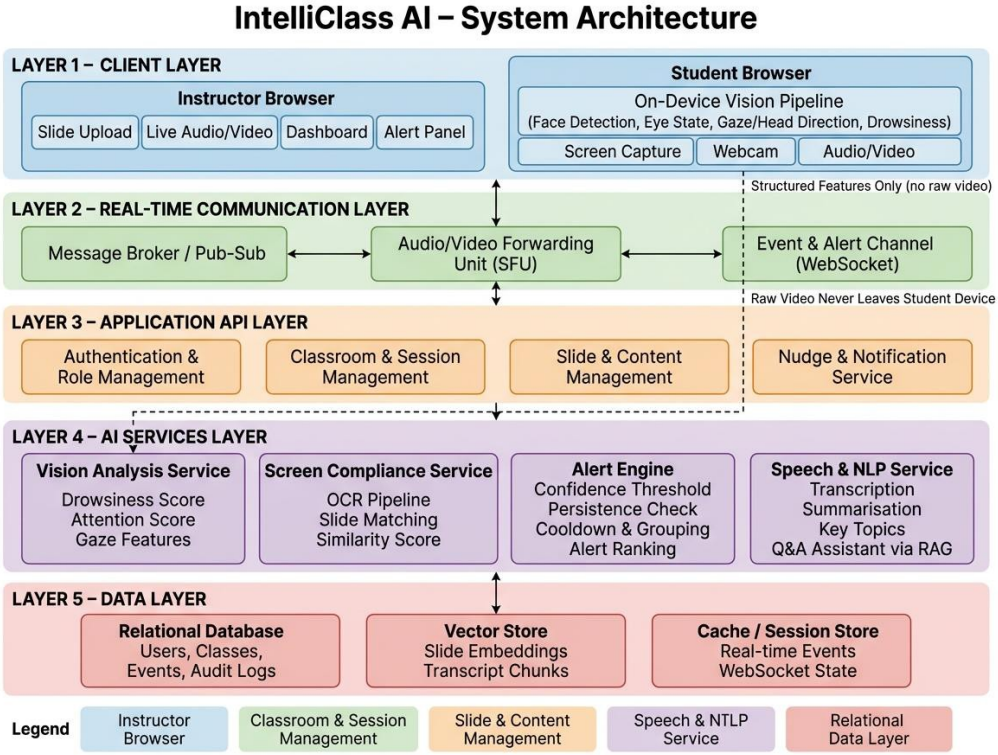


Figure 5: Block diagram of the proposed system

7.7 Evaluation Plan

Component	Evaluation Metrics
Drowsiness and attention models	Accuracy, precision, recall, F1-score, confusion matrix
Screen compliance	Precision, recall, F1-score, false-positive rate
Alert engine	Alert latency, false-alert rate, detection rate

Component	Evaluation Metrics
Private nudge	Delivery success rate and latency, and confirmation that no other participant receives it
Speech-to-text	Word Error Rate (WER)
Lecture assistant	Answer correctness, groundedness, retrieval relevance
On-device vs. server inference	Latency, CPU and bandwidth usage per student
System and scalability	End-to-end latency and resource usage at 10, 25, 50 and 100 users

Table 3: Evaluation plan

7.8 Expected Research Contribution

- A real-time, multi-signal engagement pipeline that runs on the student's device and is evaluated inside a live classroom instead of offline.
- A new, consented screen-compliance dataset and evaluation protocol that pairs student screen content with instructor slides.
- A confidence- and persistence-based alerting method judged by its false-alert rate rather than by raw detection accuracy alone.
- A measured comparison of client-side and server-side inference for online classrooms, in terms of latency, bandwidth and scalability.

8. Tools and Technology

8.1 Software

Everything we use is free and open-source, or sits inside a free tier or a free student plan. **There is no paid licence, paid API or paid hosting anywhere in the stack.**

Layer	Tools / Technologies (all free)
Frontend	Next.js, React, TypeScript, Tailwind CSS
Backend	Python, FastAPI, Pydantic
Real-time	WebRTC through LiveKit (Cloud free tier for live demos; the open-source LiveKit server self-hosted on the project VM for load tests), WebSocket, in-process event bus (Redis pub/sub only if the API runs as several replicas)
Computer Vision	MediaPipe Face Landmarker (runs in a Web Worker in the browser), OpenCV, PyTorch
OCR / Similarity	Page Visibility API baseline; tesseract.js and MiniLM sentence embeddings via transformers.js on-device; PaddleOCR, CLIP and Sentence-BERT server-side for the client-vs-server comparison experiment
Speech-to-Text	Groq free tier (Whisper large-v3, 8 hours of audio per day) for live lectures; faster-whisper (open-source Whisper small/base on CPU) as the offline fallback
LLM / RAG	Groq free tier (Llama) for live answers; small open-source LLM (Llama 3.2 3B or Qwen2.5 3B) through Ollama as the documented offline fallback; pgvector for retrieval
Database	Supabase free tier (PostgreSQL with pgvector); SQLite for local development
Frontend Hosting & Domain	Vercel free Hobby plan with a free .vercel.app subdomain (e.g. intelliclass-ai.vercel.app) and automatic HTTPS

Layer	Tools / Technologies (all free)
Backend / AI Hosting	Oracle Cloud Always Free virtual machine (Arm, 4 cores, 24 GB RAM) running the FastAPI backend and AI services in Docker; Azure for Students credit from the GitHub Student Developer Pack as the fallback; Supabase holds persistent data
DevOps	Docker, Docker Compose, Nginx, Let's Encrypt (free SSL), GitHub Actions
Testing	PyTest, Vitest, Playwright, k6/Locust (load testing)
Model Training	Google Colab and Kaggle Notebooks (free GPU)
Other	Git/GitHub (with GitHub Student Developer Pack), VS Code, Figma (free education plan)

Table 4: Software tools and technologies (zero-cost stack)

The free-tier limits that matter are known in advance. LiveKit Cloud allows 5,000 participant-minutes a month, which one fifty-student lecture would largely consume, so load tests run against the self-hosted LiveKit server on the VM and the Cloud plan is kept for live demonstrations. Supabase pauses an idle free project after a week, so a scheduled health check keeps it awake during the semester.

8.2 Hardware

- The team's existing laptops and PCs: Intel Core i5/Ryzen 5 or above, at least 8 GB of RAM (16 GB recommended for running the local LLM).
- No dedicated GPU has to be bought; model training uses the free GPUs on Google Colab and Kaggle.
- Built-in laptop webcams and microphones for testing, and team members' own devices for the concurrent-user tests.
- Free infrastructure for deployment: Vercel (frontend), an Oracle Cloud Always Free VM (backend, AI services and the self-hosted LiveKit server for load tests), LiveKit Cloud (audio/video for demos) and Supabase (data), so hosting costs nothing.

Estimated project cost: Rs. 0. Every tool and service listed above is free or open-source, so the project needs no budget.

9. Mile Stone (Gantt Chart)

#	Milestone	Main Tasks	Duration
1	Requirement Analysis & Literature Review	Requirements, literature, datasets	5 weeks
2	System Design	Architecture, database schema, UI mock-ups	4 weeks
3	Environment Setup, Auth & Roles	Repo, Docker, database, login/RBAC	3 weeks
4	Virtual Classroom	WebRTC/SFU, WebSocket, dashboards	6 weeks
5	Vision Module	Face, eye, drowsiness, gaze; model training	9 weeks
6	Screen Compliance	Sampling, OCR, slide matching	6 weeks
7	Alert Engine & Dashboard	Thresholds, cooldown, alerts, private nudge	5 weeks
8	Speech, NLP & RAG Assistant	Whisper, summaries, questions, RAG	7 weeks

#	Milestone	Main Tasks	Duration
9	Integration & Deployment	Service integration, deployment on Vercel and the project VM	3 weeks
10	Testing & Evaluation	Model metrics, load tests, user testing	5 weeks
11	Documentation & Final Demo	Final report, user manual, demonstration	4 weeks

Table 5: Project milestones (several tasks overlap)

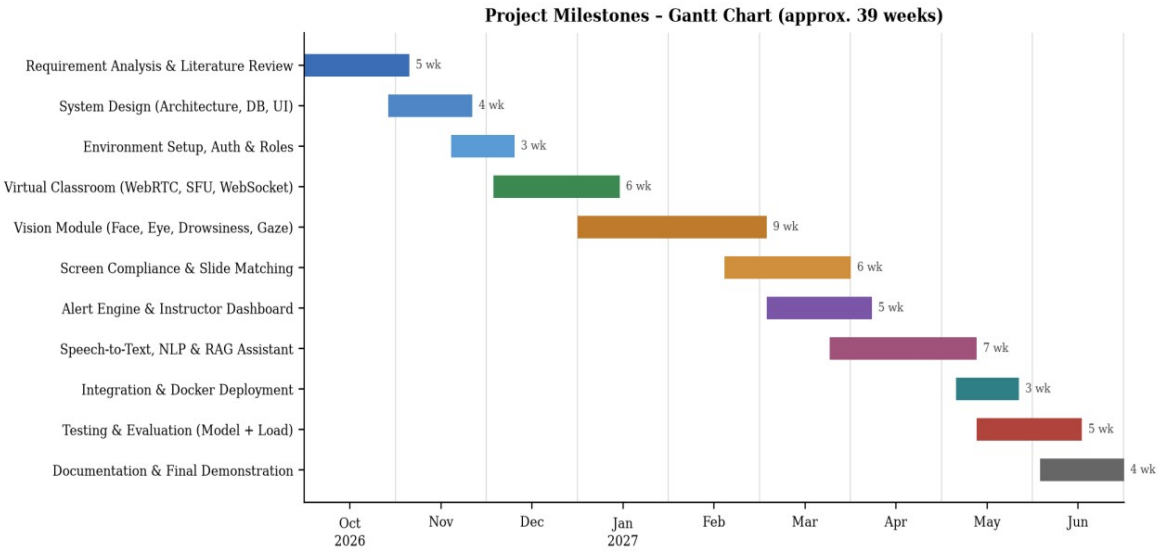


Figure 6: Gantt chart of the project timeline (October 2026 – June 2027)

10. References

[1] Zoom Video Communications. (2025). *Zoom Workplace*. https://zoom.us/

[2] Google. (2025). *Google Meet*. https://meet.google.com/

[3] Microsoft. (2025). *Microsoft Teams for Education*. https://www.microsoft.com/education/products/teams

[4] Proctorio. (2025). *Proctorio: Automated remote proctoring*. https://proctorio.com/

[5] Coghlan, S., Miller, T., & Paterson, J. (2021). Good proctor or "big brother"? Ethics of online exam supervision technologies. *Philosophy & Technology, 34*(4), 1581–1606.

[6] Whitehill, J., Serpell, Z., Lin, Y.-C., Foster, A., & Movellan, J. R. (2014). The faces of engagement: Automatic recognition of student engagement from facial expressions. *IEEE Transactions on Affective Computing, 5*(1), 86–98.

[7] Gupta, A., D'Cunha, A., Awasthi, K., & Balasubramanian, V. (2016). DAiSEE: Towards user engagement recognition in the wild. *arXiv preprint arXiv:1609.01885*. https://arxiv.org/abs/1609.01885

[8] Kaur, A., Mustafa, A., Mehta, L., & Dhall, A. (2018). Prediction and localization of student engagement in the wild. In *2018 Digital Image Computing: Techniques and Applications (DICTA)* (pp. 1–8). IEEE.

[9] Dewan, M. A. A., Murshed, M., & Lin, F. (2019). Engagement detection in online learning: A review. *Smart Learning Environments, 6*(1), Article 1.

[10] Soukupová, T., & Čech, J. (2016). Real-time eye blink detection using facial landmarks. In *Proceedings of the 21st Computer Vision Winter Workshop*. Rimske Toplice, Slovenia.

[11] Zhang, X., Sugano, Y., Fritz, M., & Bulling, A. (2015). Appearance-based gaze estimation in the wild. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4511–4520). https://arxiv.org/abs/1504.02863

[12] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.-L., Yong, M. G., Lee, J., Chang, W.-T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*. https://arxiv.org/abs/1906.08172

[13] Kartynnik, Y., Ablavatski, A., Grishchenko, I., & Grundmann, M. (2019). Real-time facial surface geometry from monocular video on mobile GPUs. *arXiv preprint arXiv:1907.06724*. https://arxiv.org/abs/1907.06724

[14] Du, Y., Li, C., Guo, R., Yin, X., Liu, W., Zhou, J., Bai, Y., Yu, Z., Yang, Y., Dang, Q., & Wang, H. (2020). PP-OCR: A practical ultra lightweight OCR system. *arXiv preprint arXiv:2009.09941*. https://arxiv.org/abs/2009.09941

[15] Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning* (pp. 8748–8763). PMLR. https://arxiv.org/abs/2103.00020

[16] Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing* (pp. 3982–3992). Association for Computational Linguistics. https://arxiv.org/abs/1908.10084

[17] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2023). Robust speech recognition via large-scale weak supervision. In *Proceedings of the 40th International Conference on Machine Learning* (pp. 28492–28518). PMLR. https://arxiv.org/abs/2212.04356

[18] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*, 9459–9474. https://arxiv.org/abs/2005.11401

[19] Chen, X., Xie, H., Zou, D., & Hwang, G.-J. (2020). Application and theory gaps during the rise of artificial intelligence in education. *Computers and Education: Artificial Intelligence, 1*, 100002.